

Guía Completa: Fundamentos de la Programación

Lógica, Algoritmos y Bases del Lenguaje C++

1. Lógica y Pensamiento de Programación

La programación es el proceso de diseñar instrucciones para que una computadora realice tareas específicas, automatice procesos y optimice recursos. Para lograrlo, es indispensable desarrollar el **pensamiento lógico**, el cual permite resolver problemas de forma sistemática a través de cuatro etapas:

1. **Análisis del problema:** Comprender a fondo qué se necesita resolver y cuáles son los requisitos.
2. **Descomposición:** Dividir el problema complejo en subproblemas más pequeños y manejables.
3. **Diseño de la solución:** Planificar paso a paso cómo resolver cada subproblema (creación del algoritmo).
4. **Implementación:** Traducir el plan a código en un lenguaje de programación y probarlo.

2. Algoritmos y Representación

Un **algoritmo** es un conjunto de pasos para resolver un problema. Para ser válido, debe cumplir con cuatro características fundamentales: ser **finito** (tener un inicio y un fin), **definido** (producir el mismo resultado ante las mismas entradas), **secuencial** (procesarse uno a la vez de forma lineal) y **preciso** (sin ambigüedades). Todo algoritmo tiene tres partes: **Entrada, Proceso y Salida**.

Herramientas de Planificación

- **Diagramas de flujo:** Representaciones gráficas con símbolos específicos (óvalos para inicio/fin, paralelogramos para entrada/salida, rectángulos para procesos y rombos para decisiones).
- **Pseudocódigo:** Una descripción informal escrita en lenguaje natural estructurado.

Ejemplo de Pseudocódigo (Cálculo de precio)

```
Inicio
Escribir("Ingrese la cantidad de bolsas:")
Leer(bolsas)
Si bolsas >= 3 Entonces
    precio = 25
Sino
    precio = 35
FinSi
total = bolsas * precio
Escribir("El total a pagar es: ", total)
Fin
```

Explicación: Este algoritmo interactúa con el usuario pidiendo la cantidad de bolsas y leyéndola en una variable. Utiliza una estructura condicional (Si / Sino) para evaluar una regla: si el cliente lleva 3 bolsas o más, el precio unitario baja a 25; de lo contrario, se mantiene en 35. Finalmente, calcula el total multiplicando las bolsas por el precio y muestra el resultado.

3. Programación Estructurada

Es un paradigma que organiza el flujo de un programa utilizando estructuras de control claras:

- **Secuencia:** Ejecución de instrucciones en orden, una tras otra.
- **Selección (Condicionales):** Toma de decisiones en el código (ej. `if / else`).
- **Iteración (Repetición):** Ciclos que repiten un bloque de código mientras se cumpla una condición (ej. `for`, `while`).
- **Modularización:** División del programa en módulos o funciones para facilitar su reutilización.

4. Fundamentos del Lenguaje C++ y Estructura Básica

C++ es un lenguaje potente con reglas estrictas: es sensible a mayúsculas y minúsculas (no es lo mismo *Main* que *main*), las instrucciones deben terminar con punto y coma (;), y toda variable debe declararse antes de usarse.

Ejemplo: Estructura Básica ("Hola Mundo")

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola Mundo";
    return 0;
}
```

Explicación:

- `#include <iostream>`: Directiva que importa la biblioteca necesaria para la entrada y salida de datos.
- `using namespace std;`: Permite usar herramientas como `cout` de manera directa sin prefijos adicionales.
- `int main() { }`: Es la función principal donde comienza obligatoriamente la ejecución del programa.
- `cout <<`: Envía el texto o mensaje a la pantalla.
- `return 0;`: Indica que el programa terminó su ejecución exitosamente y sin errores.

5. Variables, Entradas y Salidas (I/O)

Las variables son espacios en memoria para guardar datos que pueden cambiar durante la ejecución. Sus nombres no deben tener espacios y no pueden coincidir con palabras reservadas del lenguaje.

Tipos de Datos Principales

Tipo de Dato	Uso Principal	Ejemplo de Valor
int	Números enteros	15, -5, 0
float / double	Números con decimales	3.14, 550.0
char	Un solo carácter	'A', '9', '@'
string	Cadenas de texto	"Hola Mundo"
bool	Valores lógicos	true (1), false (0)

Ejemplo: Interacción con el Usuario

```
#include <iostream>
using namespace std;

int main() {
    string nombre;
    int edad;

    cout << "Ingresa tu nombre: ";
    cin >> nombre;

    cout << "Ingresa tu edad: ";
    cin >> edad;

    cout << "Hola " << nombre << ", tienes " << edad << " años." << endl;
    return 0;
}
```

Explicación: Se declaran dos variables (nombre y edad). Se usa cout (canal de salida) para pedirle la información al usuario y cin >> (canal de entrada) para capturar lo que el usuario teclea y guardarlo en la memoria de cada variable respectiva. Luego, se "concatena" o une el texto fijo con las variables utilizando los operadores <<.

6. Operadores en C++

Son símbolos especiales que permiten manipular datos y tomar decisiones lógicas en la programación.

Operadores Aritméticos

Realizan operaciones matemáticas básicas con constantes y variables numéricas.

Operador	Descripción
+ / -	Suma y Resta
* / /	Multiplicación y División
%	Módulo (devuelve el residuo exacto de una división entera)
++ / --	Incremento y Decremento (suma o resta 1 a la variable directamente)

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;

    int suma = a + b;          // Resultado: 15
    int modulo = a % b;        // Resultado: 0

    cout << "Suma: " << suma << endl;
    cout << "Módulo: " << modulo << endl;
    return 0;
}
```

Operadores Relacionales

Comparan dos valores o expresiones y devuelven una respuesta lógica y booleana: Verdadero (1) o Falso (0). Son fundamentales y base para la toma de decisiones utilizando condicionales.

Operador	Descripción
== / !=	Igualdad y Desigualdad
> / <	Mayor que / Menor que
>= / <=	Mayor o igual que / Menor o igual que

```

#include <iostream>
using namespace std;

int main() {
    int edad = 20;
    double compra = 550.0;
    int productos = 8;
    string identificacion = "0801199012345";

    if (edad >= 18) {
        cout << "Edad válida para la promoción." << endl;
    }
    if (compra > 500) {
        cout << "Monto suficiente para aplicar el descuento." << endl;
    }
    if (productos <= 10) {
        cout << "Cantidad de productos dentro del limite permitido." << endl;
    }
    if (identificacion != "") {
        cout << "Identificación proporcionada." << endl;
    }
    return 0;
}

```

Explicación de las validaciones:

- **Mayor o igual que (>=):** Evalúa que la variable edad sea exactamente 18 o superior a ese número.
- **Mayor que (>):** Verifica que el monto de la compra haya superado estrictamente el umbral de los 500.
- **Menor o igual que (<=):** Garantiza que la cantidad de productos que el usuario lleva (máximo 10) no exceda el límite permitido por la tienda.
- **Desigualdad (!=):** Comprueba que la variable identificacion tenga algún contenido ingresado, es decir, que sea distinta a un texto en blanco ("").

3. Fundamentos del Lenguaje C++ y Estructura Básica

C++ es un lenguaje potente con reglas estrictas: es sensible a mayúsculas y minúsculas (no es lo mismo *Main* que *main*), las instrucciones deben terminar con punto y coma (;), y toda variable debe declararse antes de usarse.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola Mundo";
    return 0;
}
```

4. Variables y Operadores Aritméticos / Relacionales

Las variables son espacios en memoria para guardar datos (como `int`, `float`, `string`, `bool`). Se manipulan usando operadores aritméticos (+, -, *, /, %) y relacionales (==, !=, >, <).

5. Operadores Condicionales (El Operador Ternario)

El operador condicional (o ternario ? :) es una forma rápida y compacta de escribir una decisión. Trabaja con tres partes: la condición, lo que pasa si se cumple, y lo que pasa si no se cumple.

```
int edad = 17;
string resultado = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
cout << resultado;
```

Explicación: Si la edad es 18 o mayor, devuelve "Mayor de edad". En este caso, al ser 17, el resultado devuelto será "Menor de edad". Es excelente para decisiones de solo dos opciones.

6. Estructuras de Control

Una estructura de control es una instrucción que permite decidir qué parte del código se ejecuta, cuándo y cuántas veces. Hacen que el programa tome decisiones, repita tareas o cambie de dirección.

Estructuras Condicionales

- **if / else / else if:** Ejecuta un bloque si una condición es verdadera y otro si es falsa. Se puede encadenar con `else if` para múltiples condiciones.
- **switch:** Selecciona entre múltiples casos posibles según el valor numérico o carácter de una sola variable.

Estructuras Repetitivas

- **for:** Repite un bloque de código un número conocido de veces. Su estructura general es: `for (inicialización; condición; incremento)`.
- **while:** Repite el bloque de código mientras se cumpla una condición.
- **do-while:** Ejecuta el bloque de código al menos una vez antes de verificar la condición.

Estructuras de Salto

- **break:** Sale abruptamente de un ciclo o estructura `switch`, terminando su ejecución.
- **continue:** Salta a la siguiente iteración del ciclo, omitiendo el código restante de la iteración actual.
- **return:** Termina la ejecución de una función y devuelve un valor. Por ejemplo, `return 0;` indica que el programa terminó correctamente.
- **goto:** Salta a una etiqueta específica del código, aunque es poco recomendable en programación moderna.

7. Arreglos Unidimensionales

Un arreglo es como una fila de cajas en la memoria, donde cada caja puede guardar un valor del **mismo tipo** (por ejemplo, solo números enteros). Tienen un tamaño fijo tras su declaración y ocupan un espacio contiguo en memoria.

Cada elemento se identifica con un **índice que siempre empieza en 0**.

Declaración y Recorrido

```
#include <iostream>
using namespace std;

int main() {
    // Declaramos un arreglo de 5 enteros
    int numeros[5] = {10, 20, 30, 40, 50};

    // El ciclo for nos ayuda a recorrer todo el arreglo
    for (int i = 0; i < 5; i++) {
        cout << "Elemento en la posición " << i << " = " << numeros[i] << endl;
    }

    return 0;
}
```

Explicación: La computadora empieza a contar desde 0. Así que en un arreglo de tamaño 5, los índices válidos van del 0 al 4. El ciclo for es ideal para interactuar con arreglos, ya que la variable *i* recorre exactamente cada uno de los índices.